

Analyzing Operational Disruptions at the Austin Animal Shelter

April 8th, 2025



Anna Kotlan

Introduction

The purpose of this report is to **analyze the impact of external events, such as the COVID-19 pandemic, on nonprofits like animal shelters**. Measuring the extent of a disruption is important, as it **can serve as a justification for various requests or decisions**. For example, by including operational disruption statistics in funding requests, shelters can demonstrate that their difficulties are directly tied to COVID-19, thus qualifying them for pandemic-specific financial assistance. In addition, while it can be difficult to predict when disasters will occur, anticipating the severity associated with specific types of disasters can help shelters develop more effective disaster response plans for the future.

Data Preparation

Data

The animal shelter analyzed in this report is the **Austin Animal Center**, with a dataset containing information on 48,409 animals over the past eight years. It includes details about each animal's physical characteristics, reason for stay, release reason, and length of stay (based on intake and release dates). Intake and release dates are recorded by month. Dataset Source: <https://www.kaggle.com/datasets/jackdaoud/animal-shelter-analytics>.

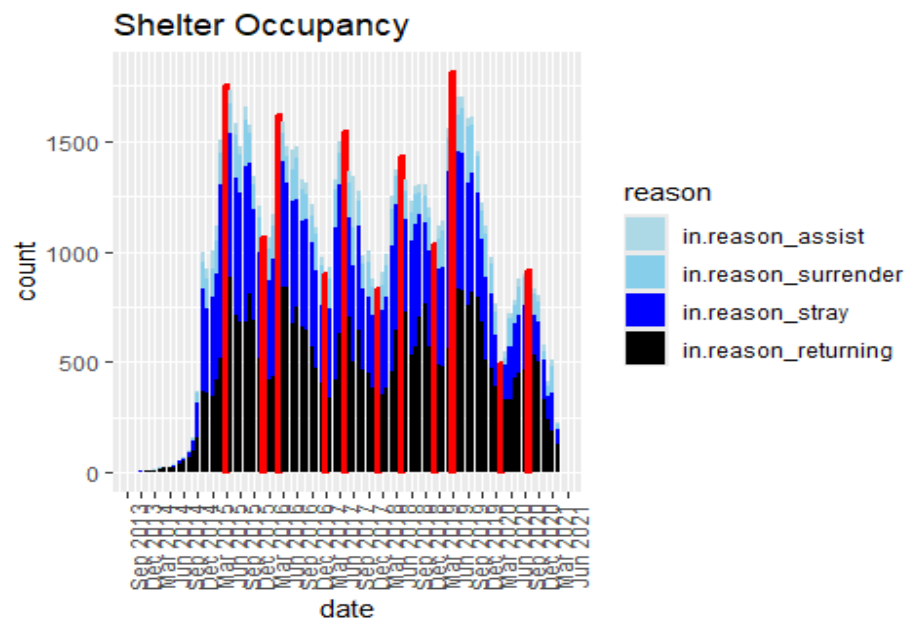
Data Transformation

There are **many ways to measure the loss an organization** has faced during a disruption, such as changes in financial performance, operational efficiency, or customer satisfaction. In this case, **we use shelter occupancy as the metric to assess the animal shelter's operational efficiency**, as it also correlates to factors such as funding received and the shelter's capacity to deliver care. To analyze shelter occupancy rather than individual animal data, the dataset was pivoted so that each row represented a date instead of an animal.

Data Exploration

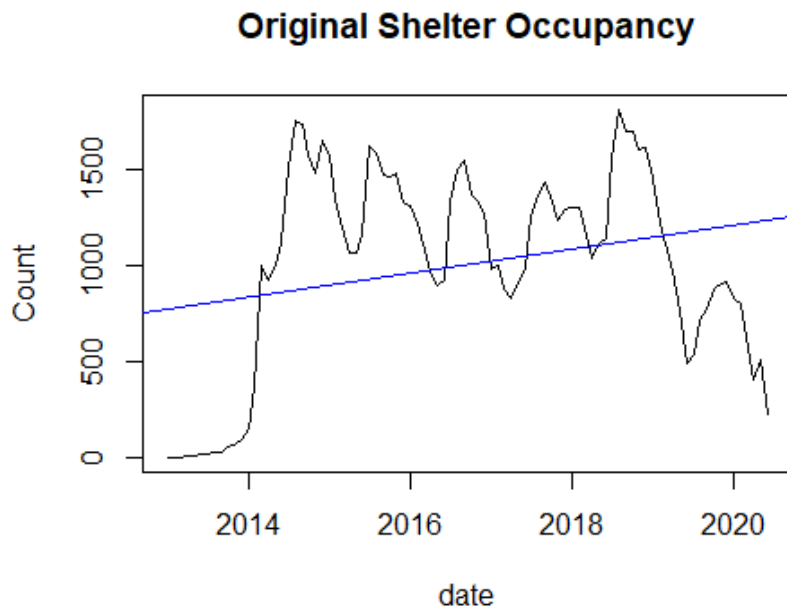
Bar Chart Plot

We can begin by using visual inspection to observe the occupancy patterns at the animal shelter. The stacked bar chart displays the occupancy of the shelter over time, color coded by if they were a returning animals (black), or new animals (various shades of blue), and their reason for entry. The peaks and dips are boxed in red. **As we can see, shelter occupancy is the highest between the months of May - July, and lowest between the months of February - April. This can be described as a seasonal cycle.**



Time Series Plot

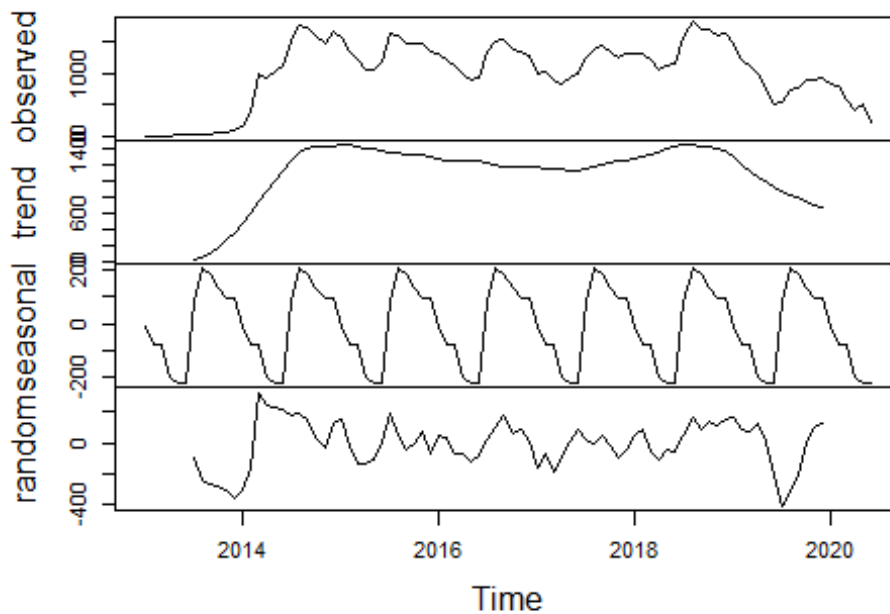
The dataframe can be transformed into a time series object for more effective time series analysis.



Isolating Time Series Components

As previously mentioned, shelter occupancy appears to be seasonal. Decomposing the time series confirms this, with the **seasonal component remaining consistent** across months. The trend component reveals a sharp initial increase, attributed to the shelter's recent opening, followed by a plateau, and then a **slight decline, attributed to the operational changes of reducing new animal intake due to the COVID-19 pandemic**. An **additive decomposition** was used, as the seasonal fluctuations remain approximately constant in magnitude throughout the time series.

Decomposition of additive time series



Model Fitting

Adjusting for Structural Breaks

The **initial intent of this report was to build an ARMA model that could help animal shelters forecast upcoming occupancy** levels to ensure readiness. This model was chosen because **simpler, univariate models such as ARMA (Autoregressive Moving Average) are often more effective for short-term forecasting** compared to more complex, multivariate models such as VAR (Vector Autoregression). However, given the **structural break caused by the COVID-19 pandemic**, removing data prior to March 2020 helps ensure the ARMA model reflects a stable time series. **Given these circumstances, it may be more appropriate to shift the research focus** to ask: What would future occupancy at the animal shelter have looked like if the pandemic had never occurred? This analysis is relevant for estimating the operational losses due to the pandemic.

Stationarity

An ARMA model relies on the assumption of stationarity - statistical properties such as mean, variance, and autocorrelation remain constant over time. Using the Augmented Dickey-Fuller test, we find evidence that the **original data is not stationary**:

Hypothesis Test:

- Null Hypothesis: time series has a unit root

- Alternative Hypothesis): time series is stationary
- p-value (0.3949) > .05 → fail reject null → time series is NOT stationary

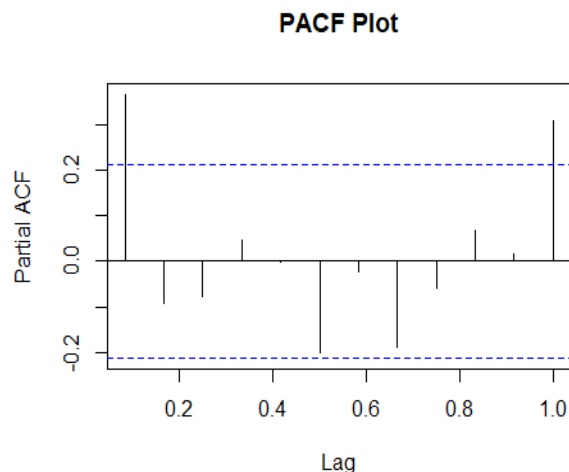
However, after **applying first-order differencing, the series becomes stationary** and is suitable for ARIMA modeling, where I represents the differencing term.

Hypothesis Test:

- Null Hypothesis: time series has a unit root
- Alternative Hypothesis): time series is stationary
- p-value (0.01) < .05 → reject null → time series is stationary at $d = 1$

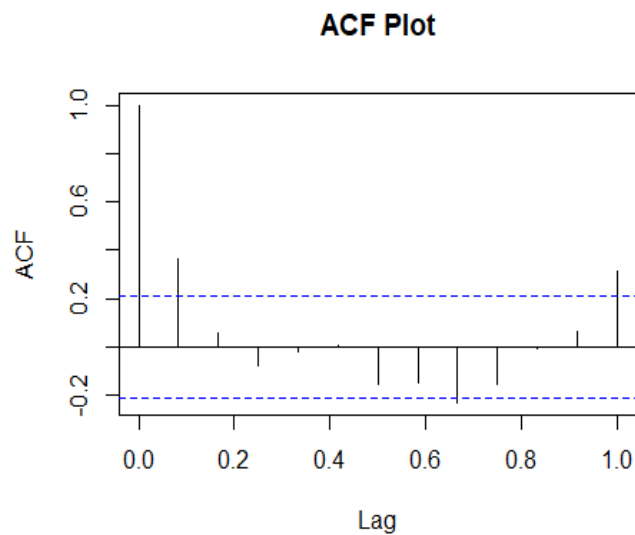
AR Component

In addition to the I component, an ARIMA model also contains an AR (AutoRegressive) component. **The AR (p) term models the relationship between the current value and its lagged values**, where p is the number of lags. The Partial Autocorrelation Function (PACF) plot shows that the **1st and 12th lags are statistically significant**. The 1st lag indicates that the current value depends on the value from one month ago, while the 12th lag suggests that the series exhibits seasonality.



MA Component

In addition to the AR component, an ARIMA model also contains an MA (Moving Average) component. **The MA (q) term models the relationship between the current value and its past forecast errors**, where q is the number of lags. The Autocorrelation Function (ACF) plot shows that **the 1st and 12th lags are statistically significant**. While the 1st lag is just shy of the significance threshold, it indicates that the current value may depend on the forecast errors from one month ago. The 12th lag suggests that the series' forecast errors exhibits seasonality.



Candidate Models

In addition to identifying the non-seasonal orders through the ADF test, ACF, and PACF plots, the data also suggests the need for a seasonal component. As such, a **SARIMA (Seasonal AutoRegressive Integrated Moving Average) model is appropriate**. The seasonal order includes a seasonal autoregressive term, seasonal differencing, and a seasonal moving average term, all with a period of 12 (indicating monthly data with yearly seasonality). **The seasonal orders were selected based on the advice to match the general structure of the non-seasonal components.**

Hence, based on the identified non-seasonal and seasonal orders, the following models are considered as candidates:

- Model 1: SARIMA(1,1,1)(1,1,1)[12]
- Model 2: SARIMA(1,1,0)(1,1,1)[12]

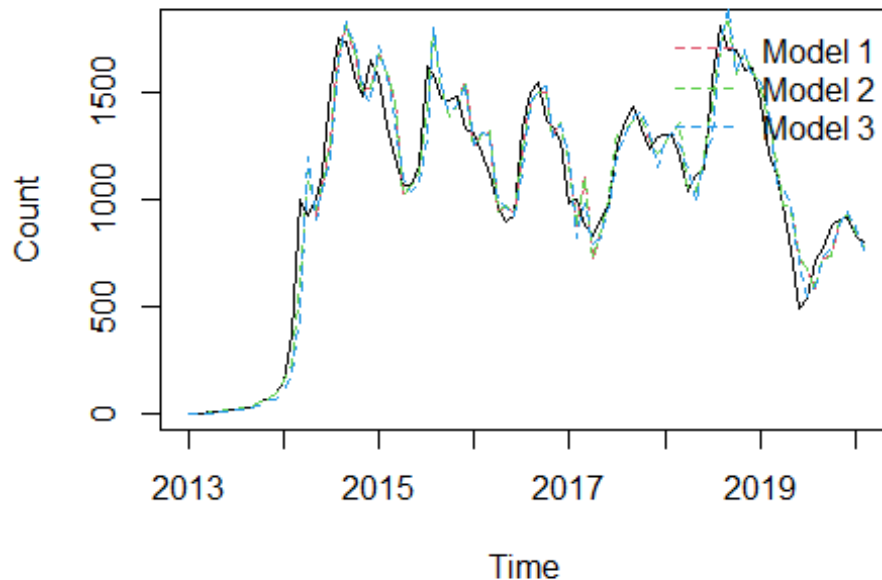
In addition to manually specifying the non-seasonal and seasonal orders, ARIMA has an `auto.arima` function that automatically selects the best model, recommending this:

- Model 3: SARIMA(1,1,0)(2,1,1)[12]

Candidate Models Plot

The following plot displays the three candidate models compared to the data used for training. As can be seen, they all follow the actual data points quite closely.

Fitted vs. Actual Occupancy



Model Selection

To select the best fit model, metrics such as AIC (Akaike Information Criterion) and BIC (Bayesian Information Criterion) are used for comparison, taking into account both the goodness of fit and model complexity. Lower AIC and BIC values indicate a better model.

Model 2 exhibits the lowest AIC and BIC values.

Model 1: AIC: 947.3267 BIC: 958.7789

Model 2: AIC: 945.3407 BIC: 954.5025

Model 3: AIC: 1089.242 BIC: 1099.012

Model Assumptions

Autocorrelation in Residuals

Before conducting any forecasting, one must verify that the model satisfies all underlying assumptions, as violations can compromise the validity of the results and lead to inaccurate forecasts. A SARIMA model relies on the **assumption of no autocorrelation in residuals**. Using the Ljung-Box test, we find evidence that there is no autocorrelation in the model's residuals.

Hypothesis Test:

- Null Hypothesis: no autocorrelation in the residuals (white noise)
- Alternative Hypothesis): autocorrelation in the residuals
- p-value (0.5702) > .05 → fail reject null → **no autocorrelation in the residuals**

Normally Distributed Residuals

A SARIMA model also relies on the **assumption of normally distributed residuals**. Using the Jarque Bera test, we find evidence that the model's residuals are normally distributed.

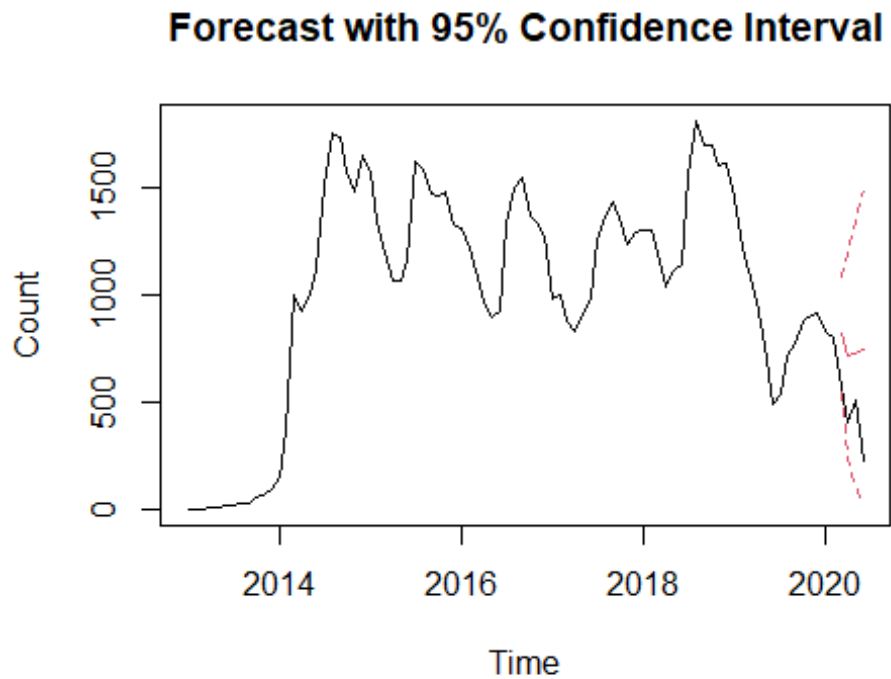
Hypothesis Test:

- Null Hypothesis: residuals are normally distributed
- Alternative Hypothesis): residuals are not normally distributed
- p-value (0.3646) > .05 → fail reject null → **residuals are normally distributed**

Forecast

The best fit model (Model 2) was used to generate forecasts. **Given that the data is recorded monthly, it is most appropriate to forecast values for the subsequent months.** Based on our analysis, had the structural break caused by COVID-19 not occurred, the estimated shelter occupancy for March through June 2020 would have been 816, 720, 723, and 749, respectively. In contrast, the actual occupancies during those months were lower: 574, 408, 506, and 227. This difference represents a **statistically significant difference**, underscoring the considerable operational impact the pandemic had on shelter capacity.

Forecast vs Observed Values Plot



Date	Point_Forecast	Lo_80	Hi_80	Lo_95	Hi_95	Actual	Difference	Percent_Loss
2020.167	816.5747	644.2533	988.8961	553.0319	1080.118	574	242.5747	29.70637
2020.25	720.832	429.7759	1011.8881	275.7002	1165.964	408	312.832	43.39874
2020.333	723.2448	334.9358	1111.5539	129.3774	1317.112	506	217.2448	30.03752
2020.417	749.0274	278.8209	1219.2339	29.9086	1468.146	227	522.0274	69.69403

Significant Difference Between Forecast vs Observed Values

Using a two-sample t-test test, we find evidence that there is a statistically significant difference between actual and forecasted occupancy.

Hypothesis Test:

- Null Hypothesis: no difference between actual and forecasted occupancy
- Alternative Hypothesis: difference between actual and forecasted occupancy

- p-value (0.01896) < .05 → reject null → there is a statistically significant difference between actual occupancy and the forecasted occupancy if the pandemic had never occurred

Conclusion

In conclusion, we can confirm that the Austin Animal Shelter experienced a statistically significant operational loss due to the onset of the COVID-19 pandemic. Four months into the pandemic, the shelter's occupancy levels were 69% lower than what would have been expected under normal operational conditions, which was predicted using an SARIMA model. Measuring the extent of such disruptions is crucial, as it can provide a basis for justifying various requests or decisions.

Thus, the Austin Animal Shelter can leverage this statistic to strengthen its case when applying for emergency or pandemic recovery grants specifically designed for those who experienced losses due to COVID-19, rather than losses driven by pre-existing operational trends.

Having experienced this disruption, the Austin Animal Shelter can develop disaster response plans to better prepare for future public health crises, recognizing that a similar magnitude of operational loss could occur again.

Code

Packages

```
library(ggplot2)
library(dplyr)
library(gridExtra)
library(reshape2)
library(lubridate)
library(tidyr)
library(MASS)
library(car)
library(xts)
library(tseries)
library(forecast)
```

Data

```
rm(list=ls())
setwd("C:/Users/Anna Kotlan/OneDrive/Documents/Bentley/4 - Spring 2025/EC 382")
dat <- read.csv("animalshelterdata.csv")
```

Data Transformation

```
# convert in.year, in.month, out.year, out.month to date objects
dat$in.date <- as.Date(paste(dat$in.year, dat$in.month, "01", sep = "-"), format = "%Y-%m-%d")
dat$out.date <- as.Date(paste(dat$out.year, dat$out.month, "01", sep = "-"), format = "%Y-%m-%d")

monthly_dates <- seq.Date(from = as.Date(cut(min(dat$in.date), "month")),
                           to = as.Date(cut(max(dat$out.date), "month")),
                           by = "month")

# dataframe for occupancy data
occupancy_dat <- data.frame(
  date = monthly_dates,
  peak_dip = NA_character_,
  in.reason_surrender = NA_real_,
  in.reason_assist = NA_real_,
  in.reason_stray = NA_real_,
  outcome_adoption = NA_real_,
  outcome_return = NA_real_,
  outcome_transfer = NA_real_
)

# peak and dip dates
peak_dates <- as.Date(c('2015-06-01', '2016-05-01', '2017-07-01', '2018-07-01',
                        '2019-06-01', '2020-10-01'))
```

```

dip_dates <- as.Date(c('2016-02-01', '2017-03-01', '2018-02-01', '2019-02-01'
, '2020-04-01'))

# Loop to fill in the occupancy data
for (i in 1:nrow(occupancy_dat)) {
  start_of_month <- occupancy_dat$date[i]
  end_of_month <- as.Date(format(start_of_month, "%Y-%m-%d")) + months(1) - d
ays(1)

  # count of records within the timeframe
  occupancy_dat$count_within_timeframe[i] <- sum(
    dat$in.date <= end_of_month & dat$out.date >= start_of_month)

  # count of records where the date matches exactly and reason is "Owner Surrender"
  occupancy_dat$in.reason_surrender[i] <- sum(
    dat$in.date == occupancy_dat$date[i] & dat$in.reason == "Owner Surrender"
)

  # count of records where the date matches exactly and reason is "Public Assist"
  occupancy_dat$in.reason_assist[i] <- sum(
    dat$in.date == occupancy_dat$date[i] & dat$in.reason == "Public Assist")

  # count of records where the date matches exactly and reason is "Stray"
  occupancy_dat$in.reason_stray[i] <- sum(
    dat$in.date == occupancy_dat$date[i] & dat$in.reason == "Stray")

  # count of records where the date matches exactly and outcome is "Adoption"
  occupancy_dat$outcome_adoption[i] <- sum(
    dat$out.date == occupancy_dat$date[i] & dat$outcome == "Adoption")

  # count of records where the date matches exactly and outcome is "Return to Owner"
  occupancy_dat$outcome_return[i] <- sum(
    dat$out.date == occupancy_dat$date[i] & dat$outcome == "Return to Owner")

  # count of records where the date matches exactly and outcome is "Transfer"
  occupancy_dat$outcome_transfer[i] <- sum(
    dat$out.date == occupancy_dat$date[i] & dat$outcome == "Transfer")
}

# assign Peak or Dip status based on dates
occupancy_dat$peak_dip <- ifelse(occupancy_dat$date %in% peak_dates, "Peak",
                                ifelse(occupancy_dat$date %in% dip_dates, "
Dip", NA))

# create in.reason difference column

```

```

occupancy_dat$in.reason_returning <- occupancy_dat$count_within_timeframe - rowSums(occupancy_dat[, grepl("in.reason", names(occupancy_dat))])

# transform data to occupancy_long_in_dat dataframe
occupancy_long_in_dat <- occupancy_dat %>%
  pivot_longer(cols = starts_with("in.reason"), names_to = "reason", values_to = "count") %>%
  dplyr::select(date, peak_dip, reason, count)

occupancy_long_in_dat$reason <- factor(occupancy_long_in_dat$reason, levels = c("in.reason_assist", "in.reason_surrender", "in.reason_stray", "in.reason_returning"))

# group occupancy_long_in_dat dataframe by date and calculating the sum of the count column
occupancy_total_dat <- occupancy_long_in_dat %>%
  group_by(date) %>%
  summarise(total_count = sum(count, na.rm = TRUE))

```

Data Exploration

Bar Chart Plot

```

ggplot(occupancy_long_in_dat, aes(x = date, y = count, fill = reason)) +
  geom_bar(stat = "identity", color = ifelse(occupancy_long_in_dat$peak_dip %in% c("Peak", "Dip"), "red", NA), linewidth = 0.8) +
  theme(axis.text.x = element_text(angle = 90, hjust = 1)) +
  scale_x_date(date_breaks = "3 month", date_labels = "%b %Y") +
  scale_fill_manual(values = c("lightblue", "skyblue", "blue", "black")) +
  labs(title = "Shelter Occupancy")

```

Time Series Plot

```

occupancy_total_ts <- ts(occupancy_total_dat$total_count, start = c(2013, 1), frequency = 12)
plot(occupancy_total_ts, xlab = "date", ylab = "Count", main = "Original Shelter Occupancy")
abline(reg = lm(occupancy_total_ts ~ time(occupancy_total_ts)), col = "blue")

```

Isolating Time Series Components

```

occupancy_total_ts_components <- decompose(occupancy_total_ts)
plot(occupancy_total_ts_components)

```

Adjusting for Structural Breaks

```

precovid_occupancy_total_ts <- ts(occupancy_total_dat$total_count, start = c(2013, 1), end = c(2020, 2), frequency = 12)

```

Stationarity

```
adf.test(precovid_occupancy_total_ts, k = 1)

##
## Augmented Dickey-Fuller Test
##
## data: precovid_occupancy_total_ts
## Dickey-Fuller = -2.4409, Lag order = 1, p-value = 0.3949
## alternative hypothesis: stationary

first_diff <- diff(precovid_occupancy_total_ts)
adf.test(first_diff, k = 1)

## Warning in adf.test(first_diff, k = 1): p-value smaller than printed p-value

##
## Augmented Dickey-Fuller Test
##
## data: first_diff
## Dickey-Fuller = -5.6461, Lag order = 1, p-value = 0.01
## alternative hypothesis: stationary
```

AR Component

```
pacf(first_diff, lag.max = 12, main = "PACF Plot") # p = 1
```

MA Component

```
acf(first_diff, lag.max = 12, main = "ACF Plot") # q = 0, 1
```

Candidate Models

```
model1 <- arima(precovid_occupancy_total_ts, order = c(1, 1, 1), seasonal = 1,
ist(order = c(1, 1, 1), period = 12))

model2 <- arima(precovid_occupancy_total_ts, order = c(1, 1, 0), seasonal = 1,
ist(order = c(1, 1, 1), period = 12))

model3 <- auto.arima(precovid_occupancy_total_ts, seasonal = TRUE, stepwise =
FALSE, approximation = FALSE)

summary(model3)

## Series: precovid_occupancy_total_ts
## ARIMA(1,1,0)(2,0,0)[12]
##
## Coefficients:
##          ar1      sar1      sar2
##          0.334  0.2100  0.2411
## s.e.    0.102  0.1012  0.1304
```

```
##
## sigma^2 = 19712: log likelihood = -540.62
## AIC=1089.24 AICc=1089.74 BIC=1099.01
##
## Training set error measures:
##           ME      RMSE      MAE      MPE      MAPE      MASE      A
CF1
## Training set 1.428937 137.096 98.36335 2.874314 12.00214 0.2260103 0.02263
457
```

Candidate Models Plot

```
ts.plot(precovid_occupancy_total_ts,
        main = "Fitted vs. Actual Occupancy",
        ylab = "Count",
        xlab = "Time")

model1_fit <- precovid_occupancy_total_ts - residuals(model1)
points(model1_fit, type = "l", col = 2, lty = 2, lwd = 1.5)

model2_fit <- precovid_occupancy_total_ts - residuals(model2)
points(model2_fit, type = "l", col = 3, lty = 2, lwd = 1.5)

model3_fit <- precovid_occupancy_total_ts - residuals(model3)
points(model3_fit, type = "l", col = 4, lty = 2, lwd = 1.5)

legend("topright",
       legend = c("Model 1", "Model 2", "Model 3"),
       col = c(2, 3, 4),
       lty = 2,
       lwd = 1.5,
       bty = "n")
```

Model Selection

```
cat("Model 1: AIC:", AIC(model1), "BIC:", BIC(model1), "\n")
## Model 1: AIC: 947.3267 BIC: 958.7789

cat("Model 2: AIC:", AIC(model2), "BIC:", BIC(model2), "\n")
## Model 2: AIC: 945.3407 BIC: 954.5025

cat("Model 3: AIC:", AIC(model3), "BIC:", BIC(model3), "\n")
## Model 3: AIC: 1089.242 BIC: 1099.012
```


Model Assumptions

Autocorrelation in Residuals

```
Box.test(residuals(model2), lag = 12, type = "Ljung-Box")

##
## Box-Ljung test
##
## data: residuals(model2)
## X-squared = 10.523, df = 12, p-value = 0.5702
```

Normally Distributed Residuals

```
jarque.bera.test(residuals(model2))

##
## Jarque Bera Test
##
## data: residuals(model2)
## X-squared = 2.0177, df = 2, p-value = 0.3646
```

Forecast

Forecast vs Observed Values Plot

```
ts.plot(occupancy_total_ts, main = "Forecast with 95% Confidence Interval", y
lab = "Count")
forecast <- predict(model2, n.ahead = 4)$pred
forecast_se <- predict(model2, n.ahead = 4)$se
points(forecast, type = "l", col = 2)
points(forecast - 2*forecast_se, type = "l", col = 2, lty = 2)
points(forecast + 2*forecast_se, type = "l", col = 2, lty = 2)
```

Forecast vs Observed Values

```
forecast <- forecast(model2, h = 4) # forecast
actual <- window(occupancy_total_ts, start = c(2020, 3), end = c(2020, 6)) #
actual values

# convert to dataframe
forecast_dat <- data.frame(
  Date = time(forecast$mean),
  Point_Forecast = as.numeric(forecast$mean),
  Lo_80 = as.numeric(forecast$lower[, 1]),
  Hi_80 = as.numeric(forecast$upper[, 1]),
  Lo_95 = as.numeric(forecast$lower[, 2]),
  Hi_95 = as.numeric(forecast$upper[, 2]))

# add actual values and difference columns
```

```

forecast_dat$Actual <- as.numeric(actual)
forecast_dat$Difference <- forecast_dat$Point_Forecast - forecast_dat$Actual

# add percentage loss column
forecast_dat$Percent_Loss <- ((forecast_dat$Point_Forecast - forecast_dat$Actual) / forecast_dat$Point_Forecast) * 100

print(forecast_dat)

```

Significant Difference Between Forecast vs Observed Values

```

t.test(forecast_dat$Actual, forecast_dat$Point_Forecast)

##
## Welch Two Sample t-test
##
## data: forecast_dat$Actual and forecast_dat$Point_Forecast
## t = -4.1168, df = 3.5218, p-value = 0.01896
## alternative hypothesis: true difference in means is not equal to 0
## 95 percent confidence interval:
## -554.15835 -93.18114
## sample estimates:
## mean of x mean of y
## 428.7500 752.4197

```